Agentic Blackbox Fuzzing of a C TOML Parser: Steering an LLM Generator Without Coverage Feedback

Jerin Anan Proma
jarinanan666@gmail.com
https://github.com/Jerinanan27/toml-agentic-fuzzing

I. Introduction

Configuration parsers written in C accept untrusted input, and C fails quietly, so a memory error can leave a program running with corrupt state. Fuzzing generates many inputs and watches for misbehavior. Random bytes work badly, since a parser rejects noise within the first few instructions, while inputs generated from the format's grammar get past that check and can still break something inside.

In this experiment, an LLM writes the generator, reading a formal grammar, producing a Hypothesis [1] strategy, and rewriting that strategy over five iterations from measured feedback. The target is `tomlc99`[1] at commit `29076df`, a C parser for TOML of roughly 2,500 lines.

The setting is blackbox: no coverage instrumentation, only exit status, stderr and wall-clock time. The question is therefore not whether an LLM can write a fuzzer, but what to measure so that the rewrites move toward inputs that find bugs. That measurement is the proxy signal. Stating in one line that depth took priority over acceptance rate changed crash yield from near zero to 121 per round, with model and iteration count unchanged.

II. Design

A. Grammar and the library gap

The grammar is the ANTLR `grammars-v4` TOML lexer and parser, copied into the repository so upstream changes cannot shift the build. The rules come to 23 content productions. `comment_or_nl` and `nl_or_comment` are excluded, handling whitespace rather than content, and inflating every coverage figure if counted. Three productions recurse through each other: arrays hold values, a value can be an inline table, inline tables hold key-values, key-values hold values. The input space is a tree, which `st.recursive` describes.

Gaps came from testing rather than from reading the parser. The one that mattered sits in the interface: `toml_parse` takes a NUL-terminated `char*` and no length, so an input containing `0x00` cannot be tested. Hex, octal and binary integers, underscore separators, all four datetime forms, all four string forms and empty inline tables were all accepted.

B. Harness and oracle

The harness is about 60 lines of C program: read stdin, call `toml_parse`, free the result, exit 0 for accepted, 50 for a clean rejection, 51 for untestable. Codes 1, 2 and 126–165 are avoided, being claimed by sanitizer runtimes and the shell, which would leave a memory-safety fault indistinguishable from an ordinary rejection. An embedded NUL returns 51 rather than being truncated, since a result for an input other than the one generated is worse than no result.

A Python oracle sorts each run into accepted, rejected, crash, hang (over 5 s) or harness error, and the check order carries weight. Sanitizer text on stderr is checked before the exit code, because UndefinedBehaviorSanitizer [2] prints and continues by default, exiting zero, so reading the code first would file genuine undefined behaviour as a success; the build also passes `-fno-sanitize-recover=all`. A stack-overflow depth belongs to the environment as much as to the library, so everything runs in a pinned image (`ubuntu:24.04`, `clang-18`) under `--ulimit stack=8388608` (8 MB); every depth below is relative to that limit and measured under ASan.

C. Structured generation and the proxy signal

The strategy produces node objects rather than TOML text, rendered by a separate serializer. Depth is then an exact tree property rather than a bracket count over text, which a quoted bracket would corrupt, and production coverage becomes exact because the generator records what was emitted.

The signal has four parts. Production coverage [3] stands in for code coverage: a production that never appears is parser code provably never reached. Nesting depth matters because the defect class is gated on depth. Acceptance rate guards against a generator whose inputs die at the front door. Rejection-message diversity uses the parser's own complaints, the only available view of the interior. Crash count was rejected as the steering term, being zero for most of a run and offering no direction; crash signatures still go back as context for what has already been found.

D. The loop

Each iteration seeds or refines, validates, runs 500 inputs, summarises and stops on budget. Returned code is executed in a fresh namespace and checked before adoption: the code must load, define `strategy`, and produce examples that serialize. Checking by execution rather than by reading matters, since a strategy can generate happily and fail only at serialization, mid-run. On failure the previous strategy stays and the failure is logged, a refusal to regress that kept several runs interpretable. Model-generated code runs in the same container as the harness.

III. Findings

A. The ordering of the objective decided the outcome

Five experiments were run (Table I), each a full five-iteration budget of 500 inputs per iteration. Experiments 3 and 4 form a controlled pair: same loop, model, budget and seed, differing only in whether the feedback states a priority between depth and acceptance.

Experiment 3 gave the four measurements equal weight. Both "depth too shallow" and "acceptance too low" fired every round with nothing to say which mattered more, and the model satisfied the term that moved most cheaply: acceptance rose from 0.00 to 0.18 while maximum depth fell from 110,001 to 20,001 at iteration 2 and stayed there. Below 20,000 levels only one defect class can be reached. Experiment 4 relabelled depth as primary and acceptance as secondary, raising acceptance by making shallow statements well-formed rather than by dropping deep values. Depth then held at 110,001 every round. Experiment 5's refinements all failed validation, so that loop never evolved.

[1] https://github.com/cktan/tomlc99

TABLE I
THE FIVE EXPERIMENTS, EACH A FULL FIVE-ITERATION BUDGET

Exp	Vocabulary / objective	Model	Crashes
1	no dotted keys, flat	GPT-OSS-120B	271
2	dotted keys added, flat	GPT-OSS-120B	278
3	22/23, flat	GPT-OSS-120B	10
4	22/23, depth primary	GPT-OSS-120B	142
5	22/23, depth primary	Qwen 3.6 27B	18

B. Control experiment

Four arms ran at the same budget three times each: random baseline, grammar seed, and the final strategies of experiments 3 and 4. Crash counts here are noisy, so means are reported: 0.0, 5.0, 1.3 and 6.7. Grammar-derived generation clearly beats random, the baseline finding nothing in all three runs: random bytes will not assemble 14,850 balanced brackets. The flat objective left the generator worse than the seed, 5.0 down to 1.3, depth falling from 110,000 to 20,000 every run. Refinement did not measurably improve on the seed (5.0 against 6.7, overlapping ranges); the prioritised objective kept what the flat one destroyed. The seed prompt carries the grammar, the gap, the thresholds and a depth requirement.

C. Triage and thresholds

The crashes reduce to five recursion cycles (Table II), fingerprinted by the set of target functions appearing three or more times in the stack. In a stack-overflow trace the top frames are wherever the stack ran out (`STRNDUP`, `expand`, allocator internals) and change between runs, while repeating frames name the recursion cycle. Hashing the top N frames, the usual approach, would have split one bug across several signatures.

TABLE II
DISTINCT DEFECTS AND THRESHOLDS (8,MB STACK, ASAN). INTERLEAVED CYCLES PRECLUDE A SINGLE DEPTH SEARCH.

Recursion cycle	Input class	Threshold
`parse_array`	`[[[...]]]`	∼14,850
`parse_keyval`	`a.a.a...`	∼87,270
`parse_inline_table+keyval`	`{k={...}}`	∼52,360
two mixed cycles	mixed nesting	pattern-dependent

Thresholds come from binary search, authoritative because that method alone tests both sides of the boundary: depth N crashes, $N-1$ survives. Hypothesis's shrinker reached 15,746 for arrays against 14,850, agreeing within 6%. The numbers are approximate, varying by 6 to 25 levels as exhaustion depends on memory state, and differing across classes because each recursion uses a different frame size.

Seventy-three crashes came back with no symbolicated stack, exhaustion leaving no room to print a trace. Re-running the class just past the threshold restored symbolication and showed the same mixed-recursion cycle; a separate bucket still holds those crashes, the inference coming from input shape, not the stack.

D. Vocabulary, successor, cost, and gaps

Experiment 1 could not write dotted keys. The serializer had no node for `a.a.a = 1`, so the `parse_keyval` defect stayed out of reach whatever the loop did. Triage exposed the gap, a dotted-key node was added, and experiment 2 found that defect on the next run without being told to look. A generator reaches only as far as the serializer can write. Measured afterwards, the two generators covered 10 and 14 of the 23 productions. Both counts are floors, the serializer rather than the strategy having supplied the surrounding key-value. Neither could write table headers, datetimes or quoted keys, and no metric existed then to show the gap.

Experiment 4 covered 22 of 23 and found no new kind of defect, but reached the old ones a new way, through quoted keys holding astral-plane and control characters. The defects sit in the recursive descent; code handling single tokens produced no crashes. Still under-tested: `bool_`; multi-line strings, written as fixed literals so escapes go untested; `array_table`, which never appears in a crashing input; and deep nesting through table headers rather than values.

Against **`tomlc17`**, the maintained successor, the recursion overflow is fixed, but the same inputs reach a different fault: `page_create()` returns NULL past a 1 GB cap, and the caller at `tomlc17.c:110` does not check.

Token counts come from the API's `usage` field: 33,591 for experiments 1 and 2, 24,732 for experiment 4, 24,997 for experiment 5. The total is under $0.05, about 1% of the $5 budget. Cost was never the limit; the free-tier rate of 8,000 tokens per minute was, capping output at 4,000 tokens.

IV. CHALLENGES

The tools built to measure the fuzzer kept hitting the bug the fuzzer was hunting. `depth()` was recursive at first, and a 5,000-level structure overflowed Python's own stack, so `depth()` became a loop. The same function later returned 0 for every document, generation having moved to whole files while `depth()` was never told about the new `Document` node. No error appeared, and only the impossibility of a crash at depth 0 surfaced the fault. Hypothesis pushes against deep nesting too: raising `max_leaves` from 200 to 5,000 made observed depth smaller, the library preferring small examples whatever the ceiling. Deep inputs were built instead by drawing a number and wrapping in a loop.

A broken generator found more crashes than the corrected version. Iteration 2 of experiment 4 found 121 crashes and iteration 3 found 2, at the same depth. One line differed: a double-wrapped `st.just` handed the serializer a strategy object instead of a value, putting a long malformed string at the centre of the nesting. Crash count therefore tracks payload shape, not depth alone, and a loop steered by crash count would have rewarded that mistake and punished the repair, a second reason to leave crash count out of the objective.

Acceptance rate failed exactly when needed. The idea was that a generator whose inputs are all rejected at the first byte tests nothing, which held for the random baseline: near-zero acceptance, zero crashes. Experiment 4 broke that idea with acceptance 0.00 alongside 121 crashes, the parser reading those inputs thousands of frames deep before rejecting correctly. Rejection at the first byte and rejection after 14,000 recursive calls both count as REJECTED, so the number cannot separate the two cases, and per-document measurement makes matters worse, one bad line rejecting the other fourteen. The repair is to measure acceptance only over documents holding no deep value.

The cross-model comparison did not work. Qwen 3.6 27B failed validation on all five refinements, so experiment 5 ran the seed strategy six times rather than evolving anything. Two failures were calls to Hypothesis functions that do not exist; two look like a reply cut off at the 4,000-token limit, which GPT-OSS approached but never crossed. Raising that limit meant shortening the prompt, so the confound stayed. The run does show the seed prompt working on both models, each reaching 22/23 coverage and 110,000 depth at round 0, and the gate catching all five.

Four judgment calls stand recorded. A timeout counts as a crash, with a signature from input shape since no stack trace exists. An input holding a NUL byte is untestable, not truncated. Fingerprints use repeating frames, not top frames. Each experiment is a separate five-iteration budget, reported as development history. Real code coverage would remove the ambiguity behind these results.

References

[1] D. R. MacIver et al., *Hypothesis: property-based testing for Python*. https://hypothesis.readthedocs.io/

[2] *AddressSanitizer* and *UndefinedBehaviorSanitizer*, LLVM/Clang documentation. https://clang.llvm.org/docs/

[3] A. Zeller, R. Gopinath, M. Böhme, G. Fraser, and C. Holler, *The Fuzzing Book*. https://www.fuzzingbook.org/

Appendix

Appendix: Reproducers and Artefacts

Minimal crashing inputs (8 MB stack, `clang-18`, `tomlc99@29076df`):

```
python3 -c "print('a = '+['*20000+'1'+']'*20000)"
python3 -c "print('a'+'.a'*100000+' = 1')"
python3 -c "print('a = '+'{ k = '*60000+'1'+' }'*60000)"
```

Each sits above its measured threshold, which varies by 6 to 25 levels between runs, so these reproduce reliably while Table II reports where the boundary actually lies. Repository artefacts: `GRAMMAR.md` (grammar and gaps), `SIGNAL.md` (proxy-signal design and revision), `FINDINGS.md`, `TARGET.md` (pinned environment), `prompts/seed.txt`, per-iteration logs (`logs/`), every generated strategy (`strategies/generated_exp*/`), and all crash records with full input, stderr and exit code (`crashes_exp*/`).